Salient Object Detection

End-to-End ML/DL Project — From-scratch CNN encoder–decoder

Author: Ernisa Sallahu

Framework: PyTorch | Dataset: DUTS | No pretrained weights

Final test-set result (improved model, DUTS-TE, 5,019 images)

IoU = 0.494 F1 = 0.662 MAE = 0.139

Problem & Pipeline

- **Goal: predict a pixel-level binary mask of the most visually salient object in an image.**
- Built end-to-end from scratch in PyTorch — no pretrained backbones.
- Pipeline: data loader → augmentation → CNN encoder–decoder → BCE+IoU loss → eval (IoU/P/R/F1/MAE) → demo.
- Dataset: DUTS — 8,970 train / 1,583 val / 5,019 test (official DUTS-TE held out).
- Preprocessing: resize 128×128, pixels → [0,1], masks binarised at 0.5.
- Augmentation: horizontal flip, random crop+resize, brightness ±30%.

Data loader

Augmentation

CNN encoder

CNN decoder

BCE + IoU loss

Adam, lr=1e-3

Eval + Demo

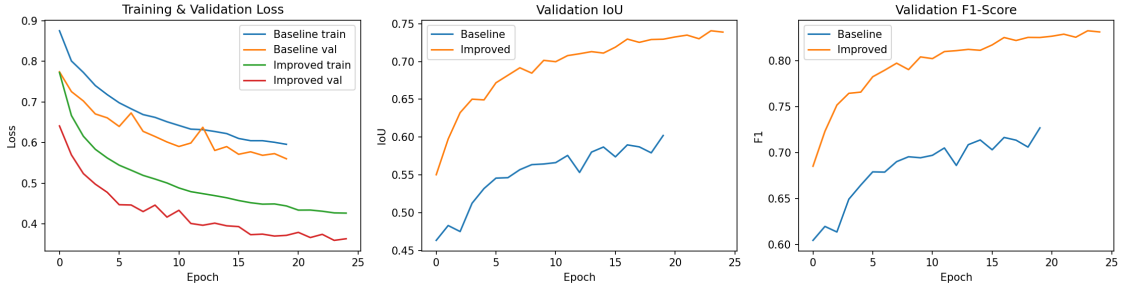
Model Architecture

Baseline (564,833 params)

- Encoder: 4 × (Conv2d → ReLU → MaxPool), channels 32→64→128→256.
- Decoder: 4 × (ConvTranspose2d, stride=2 → ReLU).
- Head: 1×1 Conv → 1-channel logits at 128×128.

Improved (1,349,633 params, ~2.4× baseline)

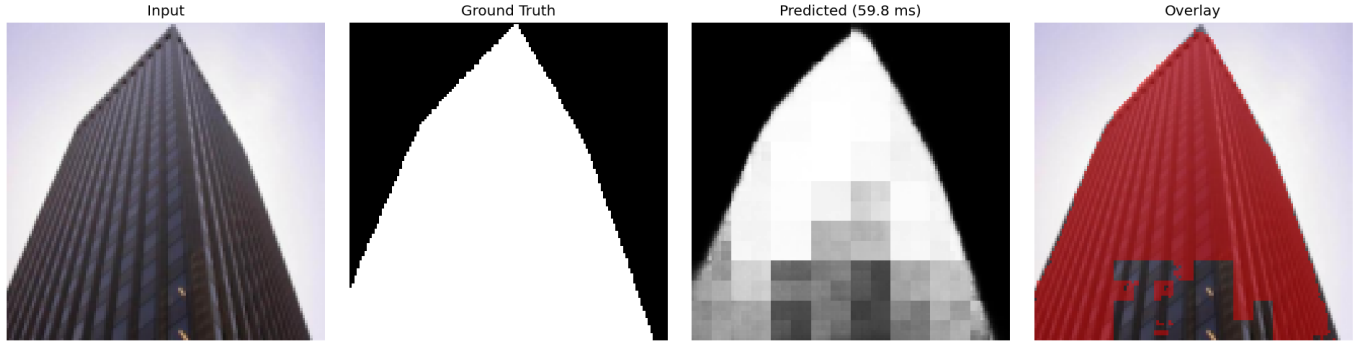
- DoubleConvBlock per stage (2 convs → BN → ReLU) — deeper feature extraction.
- BatchNorm2d after every Conv2d — stabilises training, faster convergence.
- Dropout2d(p=0.2) at the end of each encoder stage — regularisation.
- Loss: BCEWithLogits + 0.5·(1 – soft-IoU). Optimiser: Adam, lr=1e-3.
- Bonus: full save/resume — last.pt + best.pt with optimiser state.



Loss / IoU / F1 across epochs (baseline vs improved).

Results: Baseline vs Improved (DUTS-TE, 5,019 images)

Metric	Baseline	Improved	Δ
IoU	0.378	0.494	+0.117
Precision	0.470	0.589	+0.120
Recall	0.658	0.754	+0.096
F1	0.548	0.662	+0.113
MAE (↓)	0.199	0.139	-0.060
Inference	8.0 ms	10.7 ms	+2.7 ms



Improved model — input | GT | predicted mask | overlay

Takeaway

Improved model: +0.117 IoU and +0.113 F1 over baseline, ≈93 fps on Apple MPS — substantial quality gain at near-zero latency cost.

Demo & Conclusion

- **Streamlit demo (demo.py): upload an image → predicted mask + overlay + inference time.**
- Notebook demo (demo_notebook.ipynb) for offline review.
- Real-time on a Mac: ~10.7 ms / image at 128×128 on Apple MPS.

Deliverables (all completed)

- data_loader.py, sod_model.py, train.py, evaluate.py, demo.py, demo_notebook.ipynb
- Train/val/test split (70/15/15-style) on DUTS, with augmentation
- Custom CNN encoder–decoder (no pretrained weights), BCE+IoU loss, Adam lr=1e-3
- Full training loop with logging, validation, checkpointing + resume (bonus)
- Evaluation: IoU, Precision, Recall, F1, MAE + 4-panel sample visualisations
- Two experiments: BatchNorm + Dropout, and a deeper DoubleConv encoder
- Project report (PDF) and presentation slides (this deck)

Final test metrics (improved)

IoU = 0.494
F1 = 0.662
MAE = 0.139

Evaluated on 5,019 DUTS-TE images

Thank you — Questions?